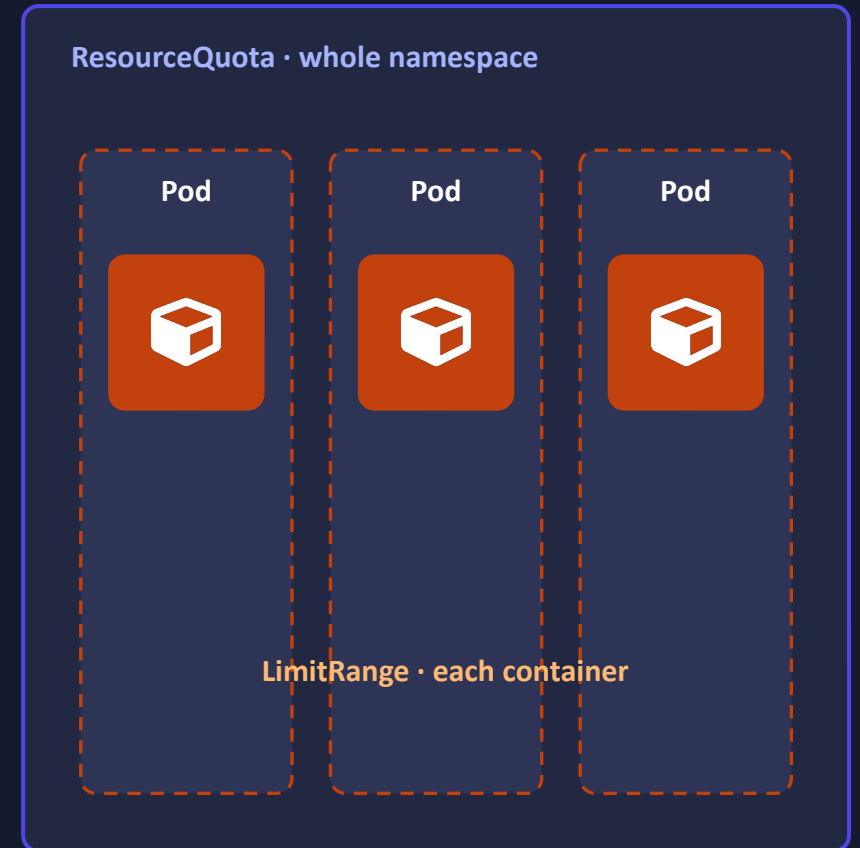


ResourceQuota & LimitRange

*Namespace budgets and per-container guardrails —
enforced by the API server at admission time*



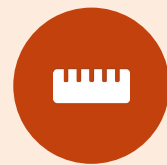
Two guardrails, two different questions



ResourceQuota

“How much can this entire namespace request and use?”

- Scope: the whole namespace
- Sums across all Pods
- Counts objects (e.g. pods: 3)
- Rejects anything that would exceed the total



LimitRange

“What rules apply to each individual container?”

- Scope: one container at a time
- Fills in default requests/limits
- Enforces per-container min / max
- Rejects containers outside the range

Both run inside the API server at admission time — before the Pod object is created and long before scheduling.



PART A

ResourceQuota

A hard resource budget for the whole namespace

The complete YAML

```
ex05a · team-quota
```

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: day07-health
spec:
  hard:
    pods: "3"

    requests.cpu: "500m"
    requests.memory: 256Mi

    limits.cpu: "1"
    limits.memory: 512Mi
```

1

namespace: day07-health

Applies only to this namespace — nothing else in the cluster.

2

pods: "3"

At most 3 Pods may exist in the namespace.

3

requests.*

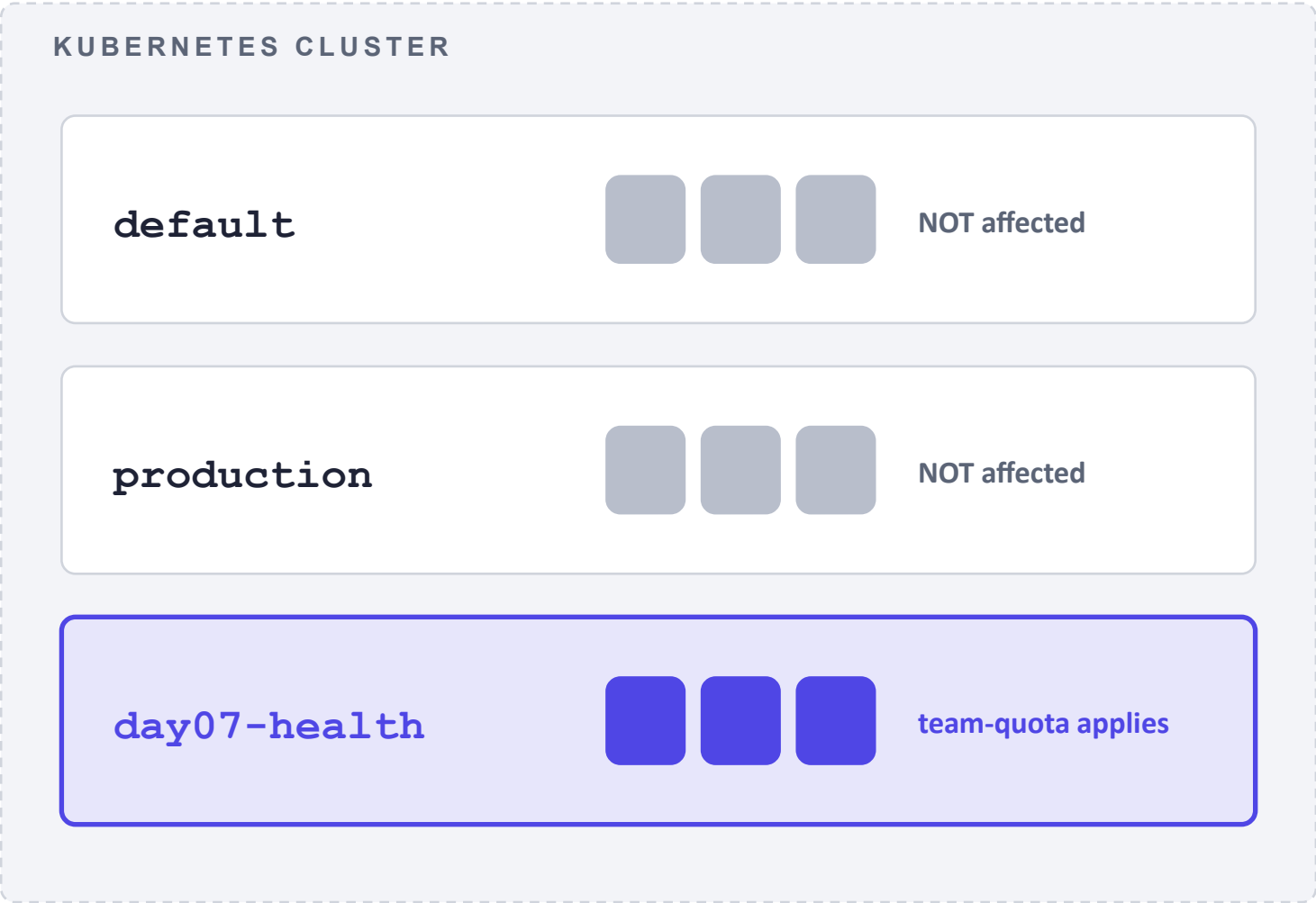
Sum of all requests: $\leq 500\text{m CPU}$ and $\leq 256\text{Mi memory}$.

4

limits.*

Sum of all limits: $\leq 1\text{ CPU}$ and $\leq 512\text{Mi memory}$.

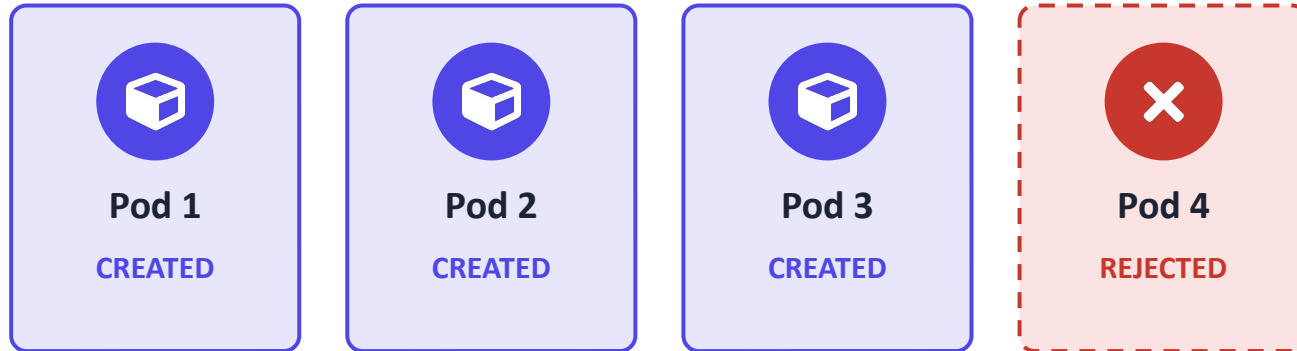
It applies to one namespace only



What the quota caps

Pods	≤ 3
CPU requests	≤ 500m
Memory requests	≤ 256Mi
CPU limits	≤ 1 CPU
Memory limits	≤ 512Mi

The 4th Pod is rejected — not Pending



Why not Pending?

The quota check happens in the API server during admission. When the namespace already has 3 Pods, the request for a 4th is refused before any Pod object exists.

No object → nothing for the scheduler to hold in Pending.

```
kubectl apply -f pod4.yaml
```

```
Error from server (Forbidden):  
exceeded quota: team-quota
```

```
requested:
```

```
  pods=1
```

```
used:
```

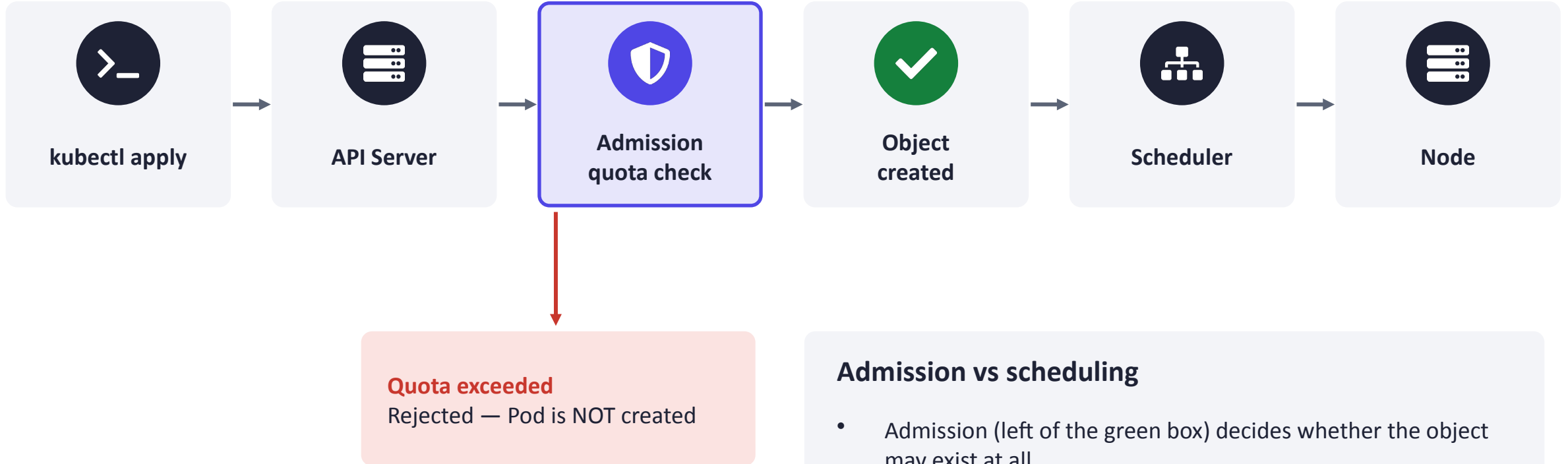
```
  pods=3
```

```
limited:
```

```
  pods=3
```

```
# Output is abbreviated –  
# real message is one line
```

What does “admission time” mean?



LimitRange also runs at this admission step — we'll see it in Part B.

Admission vs scheduling

- Admission (left of the green box) decides whether the object may exist at all.
- Scheduling (right of it) decides where an existing Pod runs.
- Quota problems are admission problems; capacity problems are scheduling problems.

DON'T MIX THESE UP

Rejected vs Pending vs OOMKilled vs Throttled

Situation	Result	Who enforces it	When
ResourceQuota exceeded	Rejected	API server (admission)	Before the Pod exists
Node doesn't have the requested CPU/memory	Pending	Scheduler — “Insufficient cpu”	Pod exists, not placed
Container exceeds its memory limit	OOMKilled	Kernel / container runtime	While running
Container exceeds its CPU limit	CPU throttling	Kernel (CFS quota)	While running

Example: a Pod requesting cpu: "10" on a 4-CPU node can pass admission (if quota allows) and then sit in Pending forever. Different problem, different fix.

RESOURCEQUOTA · REQUESTS.CPU: "500M"

Total CPU requests ≤ 0.5 CPU

500m

= 500 millicores = 0.5 CPU

1 CPU = 1000m. The quota sums the CPU requests of every container in every Pod in the namespace. Once the total reaches 500m, any Pod that adds more CPU request is rejected.

Namespace CPU requests — quota fully used (500m / 500m)



500m + 100m = 600m > 500m

→ ResourceQuota exceeded. The new Pod is rejected, even if it only wants 100m.

RESOURCEQUOTA · REQUESTS.MEMORY: 256Mi

Total memory requests $\leq$ 256 MiB

Same idea as CPU requests — the quota adds up the memory requests of every container in the namespace.

Namespace memory requests — 256Mi / 256Mi



Current total

$$64 + 64 + 128 = 256\text{Mi}$$

With the new Pod

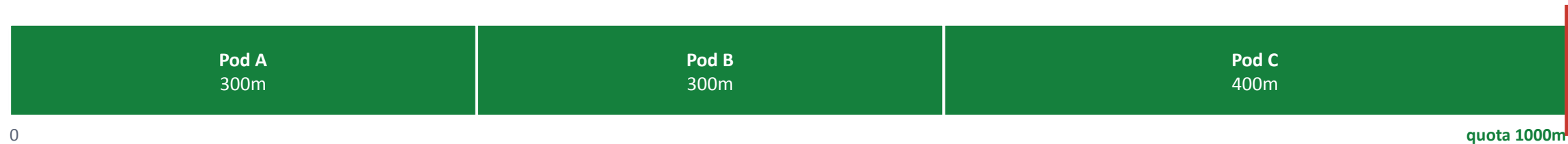
$$256 + 32 = 288\text{Mi} \rightarrow \text{Rejected}$$

Mi = mebibyte (1024 × 1024 bytes). M = megabyte (1000 × 1000). Use Mi / Gi consistently in manifests.

The limit budget is summed too

limits.cpu: "1" does NOT mean every Pod gets 1 CPU. It means the sum of all CPU limits across the namespace cannot exceed 1 CPU.

Sum of CPU limits — 1000m = 1 CPU



Sum of memory limits — 512Mi



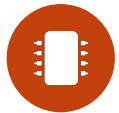
Both limit budgets are now fully used. No more limit allocation is allowed.

Two separate budgets

Namespace: day07-health

REQUEST BUDGET

What Pods reserve — used by the scheduler for placement



500m

CPU

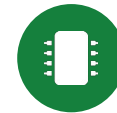


256Mi

Memory

LIMIT BUDGET

The ceilings Pods may burst up to — enforced at runtime



1 CPU

CPU



512Mi

Memory

WORKED EXAMPLE

Three Pods that exactly fill the quota

Pod	Request CPU	Request memory	Limit CPU	Limit memory
Pod A	100m	64Mi	200m	128Mi
Pod B	200m	64Mi	400m	128Mi
Pod C	200m	128Mi	400m	256Mi
Total	500m	256Mi	1000m	512Mi
Quota	500m	256Mi	1 CPU	512Mi

100%

CPU requests used

100%

Memory requests used

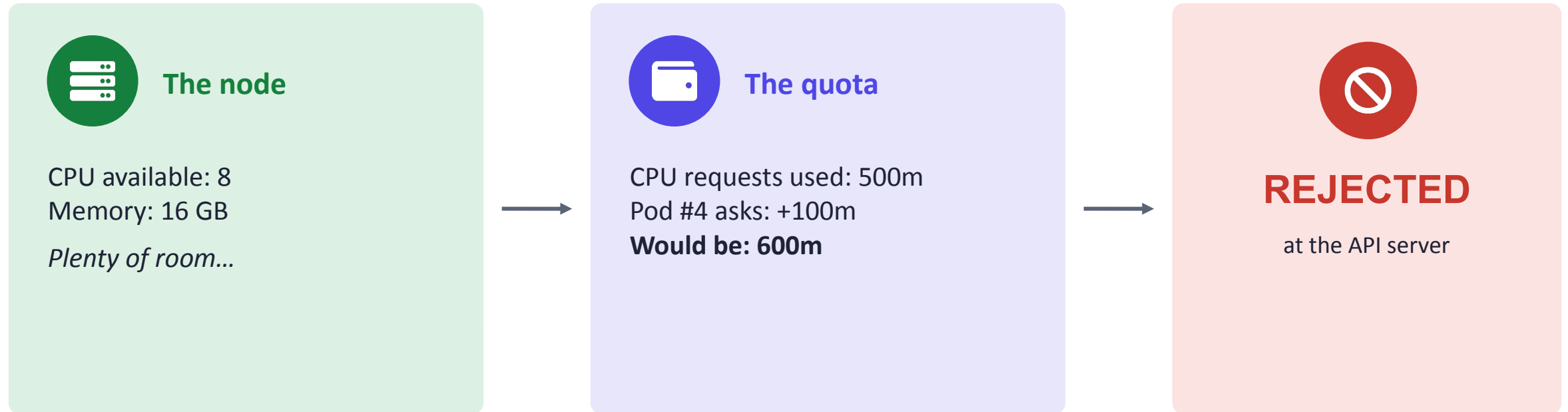
100%

CPU limits used

100%

Memory limits used

Node capacity doesn't matter here



The quota check never asks the node. It only compares the namespace's running totals with the hard limits. Here it fails twice over: CPU requests ($600\text{m} > 500\text{m}$) and the Pod count ($4 > 3$). Admission reports the violations and stops.

A guardrail for shared clusters



Without a quota

Student A creates 50 Pods

Student B creates 100 Pods

Student C requests huge memory

One team can starve everyone else.



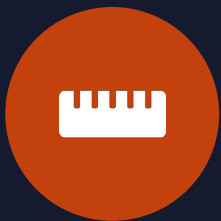
With a quota per namespace

student-01 pods ≤ 10

student-01 CPU ≤ 2

student-01 memory $\leq 4\text{Gi}$

Every namespace gets a defined, predictable budget.



PART B

LimitRange

Per-container defaults and guardrails

The complete YAML

ex05b · team-limits

```
apiVersion: v1
kind: LimitRange
metadata:
  name: team-limits
  namespace: day07-health
spec:
  limits:
    - type: Container
      default:
        cpu: 100m
        memory: 64Mi
      defaultRequest:
        cpu: 50m
        memory: 32Mi
      min: { cpu: 10m, memory: 8Mi }
      max: { cpu: 500m, memory: 256Mi }
```

1

type: Container

Rules are evaluated per container, not per Pod or namespace.

2

default

The LIMIT stamped on a container that sets no limit.

3

defaultRequest

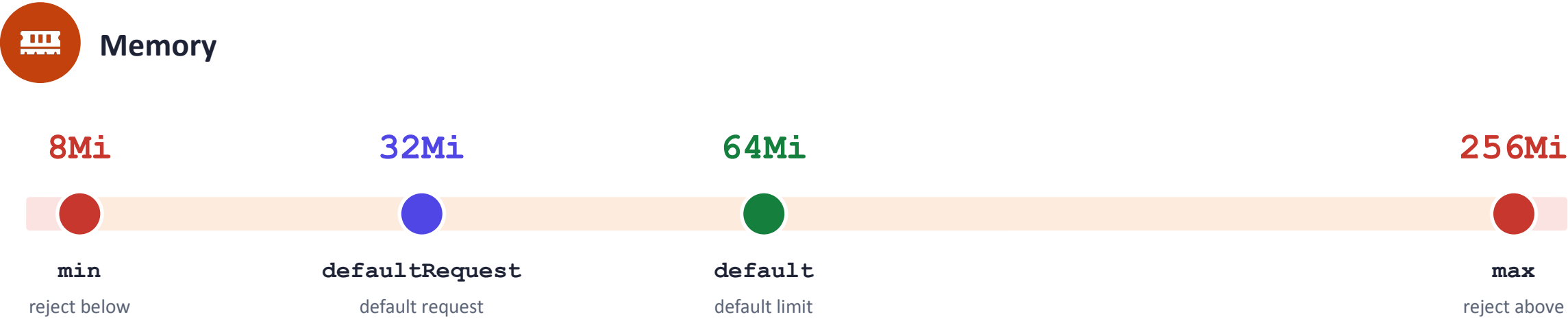
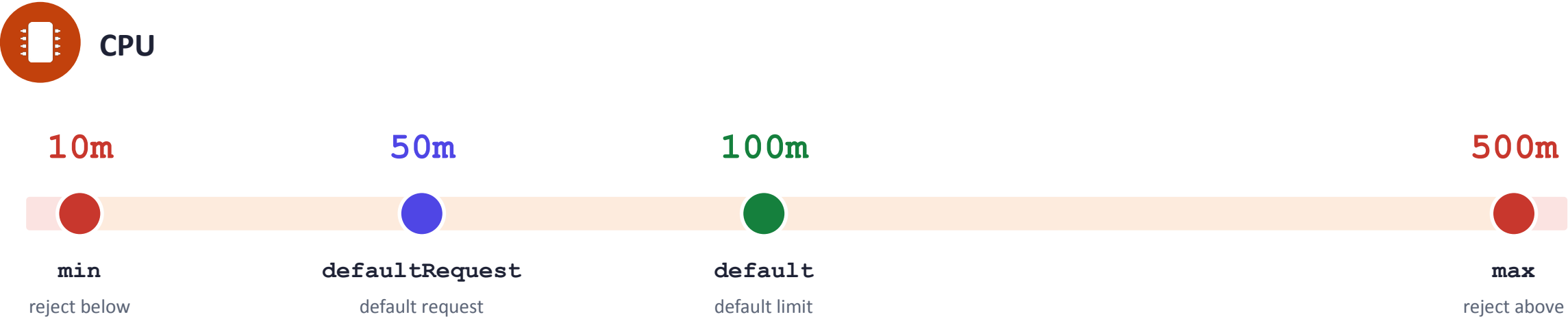
The REQUEST stamped on a container that sets no request.

4

min / max

Allowed range. Values outside it are rejected.

Where the four values sit

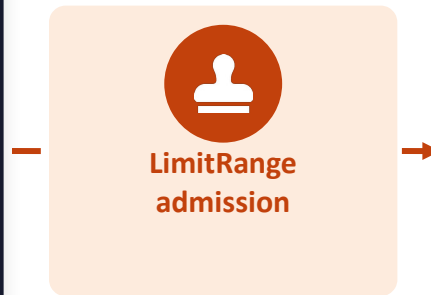


Not to scale. Shaded ends = rejected zone.

Missing values are stamped on automatically

YOU SUBMIT

```
containers:  
- name: app  
  image: nginx  
  # no resources:
```



WHAT GETS STORED

```
containers:  
- name: app  
  image: nginx  
  resources:  
    requests:  
      cpu: 50m  
      memory: 32Mi  
    limits:  
      cpu: 100m  
      memory: 64Mi
```

Check it: `kubectl get pod <name> -o yaml` — the stored spec shows the injected values. This is a mutating step.

Six scenarios for one container

#	Container specifies	After defaulting	Result	Why
1	nothing	req 50m/32Mi · lim 100m/64Mi	Accepted	Both defaults applied
2	req 20m · lim 400m (cpu)	unchanged	Accepted	10m ≤ 20m and 400m ≤ 500m
3	lim cpu 300m only	req 300m · lim 300m	Accepted	Request copies the limit when only a limit is set
4	lim cpu 800m	—	Rejected	Above max 500m
5	req memory 4Mi	—	Rejected	Below min 8Mi
6	req cpu 200m, no limit	req 200m · lim 100m	Rejected	Default limit is lower than the request

Rows 2–6 show the CPU or memory value that matters; the other resource falls back to defaults.

Setting a request without a limit

YOU SUBMIT

```
resources:  
  requests:  
    cpu: 200m  
  # no limits
```



LimitRange fills the missing limit:

`limits.cpu = 100m`

Now request 200m > limit 100m



Rejected

A request can never be larger than its limit, so validation fails with an error similar to “requests: must be less than or equal to cpu limit of 100m”.



Fix

Always set a limit too when your request is above the default limit — e.g. request 200m, limit 300m (still $\leq$ max 500m).

Per container — and only for new Pods

One Pod, two containers, no resources set

Pod: web

app

req 50m / 32Mi
lim 100m / 64Mi

sidecar

req 50m / 32Mi
lim 100m / 64Mi

Pod totals: req 100m/64Mi · lim 200m/128Mi

Each container gets its own defaults — so the Pod's quota footprint doubles.



Evaluated per container

type: Container means each container is defaulted and checked on its own. min/max are not Pod totals.



Only affects new Pods

Existing Pods are not changed when you create or edit a LimitRange. Recreate them to pick up new rules.



Also admission-time

Like ResourceQuota, a violation rejects the request at the API server — no Pending Pod.

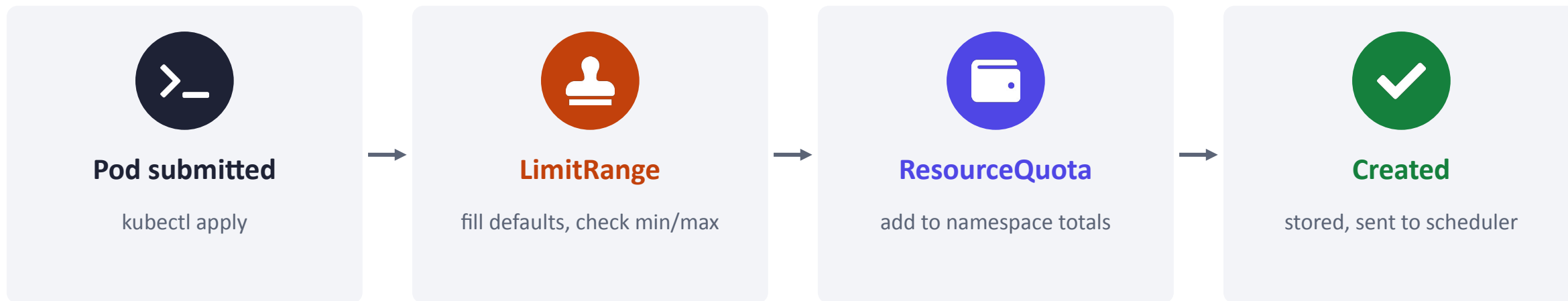


PART C

Better together

How LimitRange and ResourceQuota cooperate

LimitRange runs first, then the quota counts



Why order matters

Because defaults are applied before the quota check, the defaulted values are what gets counted against the quota.

Required fields

A quota on requests.* or limits.* forces every new Pod to declare them. Without a LimitRange, a bare Pod is rejected with “must specify limits.cpu...”-style errors.

Which limit do you hit first?

Scenario 1: bare Pods (defaults only)

Pods **3 / 3**

CPU requests **150m / 500m**

Mem requests **96Mi / 256Mi**

CPU limits **300m / 1000m**

Mem limits **192Mi / 512Mi**

Pod #4 → rejected by pods: 3, while every resource budget still has room.

Scenario 2: one max-sized Pod

Pods **1 / 3**

CPU requests **500m / 500m**

Mem requests **256Mi / 256Mi**

CPU limits **500m / 1000m**

Mem limits **256Mi / 512Mi**

Container at LimitRange max (req = lim = 500m / 256Mi). Any next Pod → rejected on requests.

Who enforces what

ResourceQuota	Whole namespace	API server, admission
LimitRange	Individual container	API server, admission
Resource requests	Placement	Scheduler
Resource limits	Runtime ceilings	Kernel / container runtime
QoS class	Eviction priority	Derived from requests + limits

LimitRange defaults here (50m request < 100m limit) produce Burstable Pods. Request = limit on every container → Guaranteed.

From kubectl apply to a running container



Admission decides if the Pod may exist. **Scheduling** decides where. **Runtime** enforces limits.

WATCH OUT

Common mistakes



Quota = per-Pod limit

ResourceQuota caps namespace totals, not individual Pods.



Expecting Pending on quota

Quota violations are rejected at admission; Pending is a scheduling problem.



Reading `default` as a request

In LimitRange, default is the default LIMIT; defaultRequest is the request.



Request above the default limit

Set a limit too, or request > default limit makes the spec invalid.



Forgetting sidecars

Every container gets defaults, so multi-container Pods count more.



Expecting old Pods to change

LimitRange and quota changes apply to new admissions only.

Lab: verify both guardrails



```
$ kubectl apply -f ex05a.yaml -f ex05b.yaml
$ kubectl describe quota team-quota \
  -n day07-health
$ kubectl describe limitrange team-limits \
  -n day07-health

# bare Pod: defaults get stamped
$ kubectl run p1 --image=nginx -n day07-health
$ kubectl get pod p1 -n day07-health \
  -o jsonpath='{.spec.containers[0].resources}'

# create p2, p3, then p4 → expect Forbidden
$ kubectl run p4 --image=nginx -n day07-health
```

What you should see



Used vs Hard columns

describe quota shows running totals



Defaults listed

describe limitrange shows min/max/defaults



p1 has resources

req 50m/32Mi, lim 100m/64Mi injected



Quota usage rises

pods, requests.*, limits.* go up per Pod



p4 is Forbidden

exceeded quota — not Pending

REMEMBER

One-line revision

ResourceQuota = namespace budget enforced by the API server at admission time.

LimitRange = per-container defaults and min/max, also enforced at admission time.

1

Quota sums requests and limits as two separate budgets, plus object counts.

2

Exceeding a quota → Rejected. Not enough node capacity → Pending.

3

default = default limit; defaultRequest = default request.

4

LimitRange defaults are applied before the quota counts them.

5

Every container gets its own defaults — sidecars add up.

Can you answer these?

Q1 Is ResourceQuota a per-Pod or per-namespace limit?

Q2 Why is the 4th Pod rejected rather than Pending?

Q3 Three Pods request 200m + 200m + 100m CPU. Can a Pod requesting 50m be added?

Q4 What is the difference between default and defaultRequest?

Q5 A container sets no resources. What does it end up with?

Q6 A container requests 200m CPU with no limit. What happens, and why?

Q7 A Pod has 2 containers and no resources set. What does it add to the quota?

Q8 Which runs first at admission: LimitRange or ResourceQuota — and why does it matter?